

Universidad De San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Laboratorio Lenguajes Formales y de Programación
Auxiliar Douglas Omar Arreola Martínez
Sección "B -"



“MANUAL DE USUARIO”

Luis Angel Barrera Velásquez

Carné: 202010223

Índice

Objetivos	3
General:	3
Específicos:	3
Introducción.....	4
Descripción de la Solución	4
Conceptos utilizados para la solución	5
Lectura de archivos de texto plano	6
Escritura de archivos.....	6
Ciclos for.....	7
Condicional if	8
Condicional if.. else	9
Variables globales	10
Analizador Léxico	11
Análisis.....	11
Tabla de Tokens.....	13
Método del Árbol	14

Objetivos

General:

Brindar al lector información acerca del desarrollo de este proyecto y como se le dio solución para una futura actualización o corrección de errores técnicos como actualizaciones del sistema.

Específicos:

- Detallar con palabras la solución presentada en el código paso a paso para seguir un orden y lógica del programa.
- Proporcionar los conceptos que fueron empleados para la solución de este proyecto con teoría para que sea más entendible la solución expresada en palabras.

Introducción

Este manual técnico esta elaborado con el propósito de detallar el funcionamiento del código fuente del proyecto y en caso de que lo lea alguien con conocimientos sobre programación sea de ayuda para realizar futuras actualizaciones sobre el programa o corrección de algún error que suceda durante la ejecución. Es importante mencionar que este proyecto se realizo una fusión entre conceptos básicos de programación como también la utilización de expresiones regulares que viene siendo mas complejo pero con la implementación de estructuras básicas como ciclos o condicionales en la programación se puede controlar la lectura del archivo solicitado por medio de un automata.

Descripción de la Solución

Para la solución de este proyecto fue necesario utilizar la lectura de archivos de texto plano y guardarlos en un buffer para realizar el respectivo análisis del archivo. Para esto fue necesario la utilización del concepto de expresiones regulares para generar a partir del método del árbol binario de sintaxis generar un autómata:

- Primero se realizó una interfaz gráfica por medio de la librería tkinter Python para que el usuario del software tenga una mejor perspectiva de la funcionalidad del software para esto se cargaron diferentes funciones en botones como cargar, analizar, reportes, salir y para cada uno de los filtros mostrándolos en un área de la ventana la imagen dependiendo que filtros fueron solicitados en el archivo.
- Se realizó para cada token una expresión regular la cual servirá para el analizar léxicamente el archivo de entrada y comprobar que no existan errores léxicos (más adelante estará detallado cada expresión regular junto con su procedimiento para obtener su autómata finito determinista)
- Para el análisis se implementó varios autómatas resumidos en uno solo a través de un ciclo for en la cual entra el archivo de prueba y lo recorre carácter por carácter ignorando espacios, tabulaciones y saltos de línea, a través de la utilización de estados que son evaluados con condicionales if, elif y else.
- El archivo debe de seguir con el autómata y al encontrar cada token fue guardado en una lista de objetos de tipo token y en caso de que se detectara un error se guarda el carácter o el lexema erróneo que causó el error en una lista de objetos de tipo error.
- Una vez teniendo una lectura exitosa todas las funcionalidades del programa funcionan de lo contrario notificara al usuario que hubo un error por lo cual deberá corregir en el archivo de entrada ayudándose del reporte html con los tokens reconocidos y los errores que se generaron durante la lectura.

Conceptos utilizados para la solución

Lectura de archivos de texto plano

La entrada y salida de ficheros (comúnmente llamada File I/O) es una herramienta que Python, como la mayoría de los lenguajes, nos brinda de forma estándar. Su utilización es similar a la del lenguaje C, aunque añade mayores características.

El primer paso para introducirnos es saber cómo abrir un determinado archivo. Para esto se utiliza la función `open`, que toma como primer argumento el nombre de un fichero.

```
1. open("archivo.txt")
```

No está de más aclarar que esto no significa ejecutar el programa por defecto para editar el fichero, sino cargarlo en la memoria para poder realizar operaciones de lectura y escritura.

En caso de no especificarse la ruta completa, se utilizará la actual (donde se encuentre el script).

Como segundo argumento, `open` espera el modo (como una cadena) en el que se abrirá el fichero. Esto indica si se utilizará para lectura, escritura, ambas, entre otras.

```
1. open("archivo.txt", "r")
```

La `r` indica el modo lectura. Si se intentara utilizar la función `write` para escribir algo, se lanzaría la excepción `IOError`. A continuación, los distintos modos.

- `r` – Lectura únicamente.
- `w` – Escritura únicamente, reemplazando el contenido actual del archivo o bien creándolo si es inexistente.
- `a` – Escritura únicamente, manteniendo el contenido actual y añadiendo los datos al final del archivo.
- `w+`, `r+` o `a+` – Lectura y escritura.

Escritura de archivos

Este concepto fue utilizado para la escritura del reporte HTML ya que fue escrito como si escribiéramos un txt pero se uso la sintaxis de HTML y guardarlo como *.html como fue solicitado en el enunciado.

```
1. f = open("archivo.txt", "w")
```

En este ejemplo he abierto el fichero archivo.txt que, en caso de no existir, será automáticamente creado en el directorio actual. A continuación escribiré algunos datos utilizando la función file.write.

```
1. f.write("Esto es un texto.")
```

Esta función fue utilizada hasta de ultimo ya que mientras la creación del html se fue concatenando una cadena bastante larga agregando cada etiqueta que se necesitara como por ejemplo:

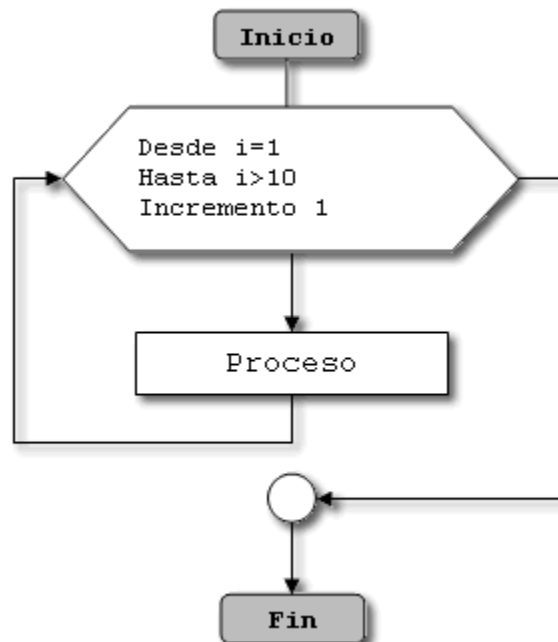
```
1. cadena+="<table border=\"1\" width=\"{str(imagen.ancha)}\" height=\"{str(imagen.alto)}\">"
```

por ultimo se cerro el archivo leído o de escritura por medio de la función close

```
1. f.close()
```

Ciclos for

El bucle for se utiliza para recorrer los elementos de un objeto iterable (lista, tupla, conjunto, diccionario, ...) y ejecutar un bloque de código. En cada paso de la iteración se tiene en cuenta a un único elemento del objeto iterable, sobre el cuál se pueden aplicar una serie de operaciones.



Condicional if

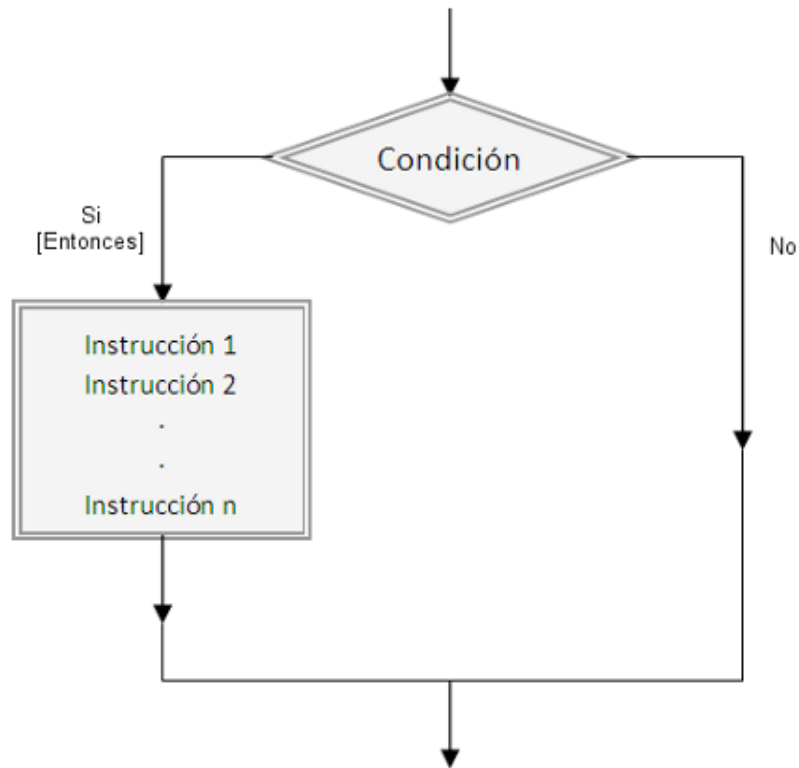
La estructura de control if ... permite que un programa ejecute unas instrucciones cuando se cumplan una condición. En inglés "if" significa "si" (condición).

La sintaxis de la construcción if es la siguiente:

```
if condición:
    aquí van las órdenes que se ejecutan si la condición es cierta
    y que pueden ocupar varias líneas
```

La ejecución de esta construcción es la siguiente:

- La condición se evalúa siempre.
 - Si el resultado es True se ejecuta el bloque de sentencias
 - Si el resultado es False no se ejecuta el bloque de sentencias.



Condicional if.. else

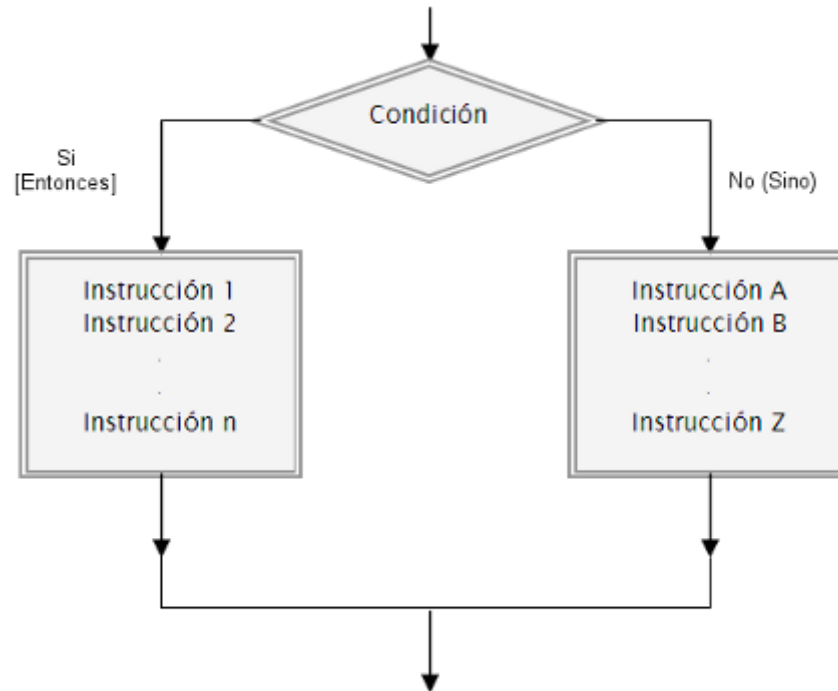
La estructura de control if ... else ... permite que un programa ejecute unas instrucciones cuando se cumple una condición y otras instrucciones cuando no se cumple esa condición. En inglés "if" significa "si" (condición) y "else" significa "si no". La orden en Python se escribe así:

La sintaxis de la construcción if ... else ... es la siguiente:

```
if condición:
    aquí van las órdenes que se ejecutan si la condición es cierta
    y que pueden ocupar varias líneas
else:
    y aquí van las órdenes que se ejecutan si la condición es
    falsa y que también pueden ocupar varias líneas
```

La ejecución de esta construcción es la siguiente:

- La condición se evalúa siempre.
 - Si el resultado es True se ejecuta solamente el bloque de sentencias 1
 - Si el resultado es False se ejecuta solamente el bloque de sentencias 2.



Variables globales

Las variables globales son las variables con alcance global. El alcance global significa que la variable es accesible desde cualquier lugar del programa. Las variables globales se declaran fuera de las funciones ya que su alcance no se limita a ninguna función. La vida útil de la variable global es igual al tiempo de ejecución del programa.

No tenemos que declarar explícitamente las variables antes de usarlas, por lo tanto, para diferenciar entre una variable local y una global, necesitamos especificar que la variable a la que estamos accediendo es la variable global o no. Podemos especificar una variable como global en Python usando la palabra clave `global`.

Si pasamos el valor a la variable global dentro de una función sin declararla variable global, el valor se pasará a la nueva variable con el mismo nombre. Y el alcance de la nueva variable estará restringido al alcance de la función.

Por ejemplo:

```
date = "17-06-2002"

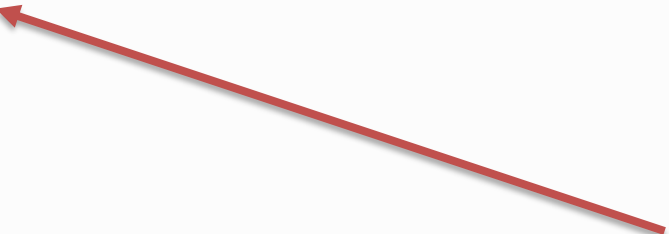
def update_date():

    global date #declaracion de la variable global hace referencia a:

    date = "12-03-2021"

update_date()

print(date)
```



Analizador Léxico

Un analizador léxico o analizador lexicográfico (en inglés scanner o tokenizer) es la primera fase de un compilador, consistente en un programa que recibe como entrada el código fuente de otro programa (secuencia de caracteres) y produce una salida compuesta de tokens (componentes léxicos) o símbolos. Estos tokens sirven para una posterior etapa del proceso de traducción, siendo la entrada para el analizador sintáctico (en inglés parser).

La especificación de un lenguaje de programación a menudo incluye un conjunto de reglas que definen el léxico. Estas reglas consisten comúnmente en expresiones regulares que indican el conjunto de posibles secuencias de caracteres que definen un token o lexema.

Análisis

Esta etapa está basada usualmente en una máquina de estados finitos. Esta máquina contiene la información de las posibles secuencias de caracteres que puede conformar cualquier token que sea parte del lenguaje (las instancias individuales de estas secuencias de caracteres son

denominados lexemas). Por ejemplo, un token de naturaleza entero puede contener cualquier secuencia de caracteres numéricos.

- Token (o componente léxico): Secuencia de caracteres con significado sintáctico propio.
- Lexema: Secuencia de caracteres cuya estructura se corresponde con el patrón de un token.
- Patrón: Regla que describe los lexemas correspondientes a un token.

Tabla de Tokens

Token	Patrón (Expresión Regular)
Titulo	$L^+ = "C(L D)+C";$
Ancho	$L^+ = "D^+";$
Alto	$L^+ = "D^+";$
Filas	$L^+ = "D^+";$
Columnas	$L^+ = "D^+";$
Celdas	$L^+ = "(=)({}((\{D+(,N(,L+(,)(\#)(L D)+\{)(,)?+\{)(;))$
Filtros	$L^+ = "L^+";$
Separador	@@@@

Donde:

$L = \{a-z, A-Z\}$

$D = \{0,1,2,3,4,5,6,7,8,9\}$

$C = \{ " \}$

Método del Árbol

Para la solución de este proyecto se utilizaron varias expresiones regulares las cuales se les realizó su respectivo autómata finito determinista para cada una de las expresiones por medio del método del árbol binario de sintaxis. A continuación, se detalla cada proceso del método de árbol para cada expresión regular:

Titulo

$L = \{a-z, A-Z\}$

$D = \{0,1,2,3,4,5,6,7,8,9\}$

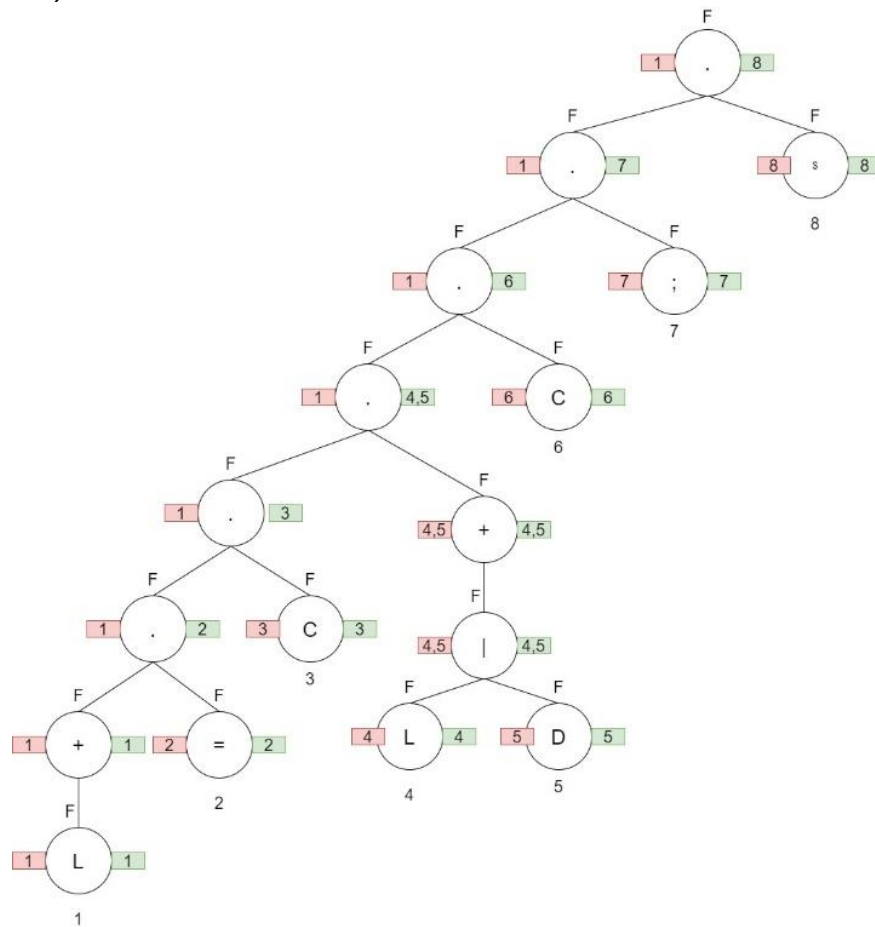
$C = \{ \text{"} \}$

Expresión Regular:

$L^+ = "C(L|D)^+C";"$

Proceso para Autómata Finito Determinista:

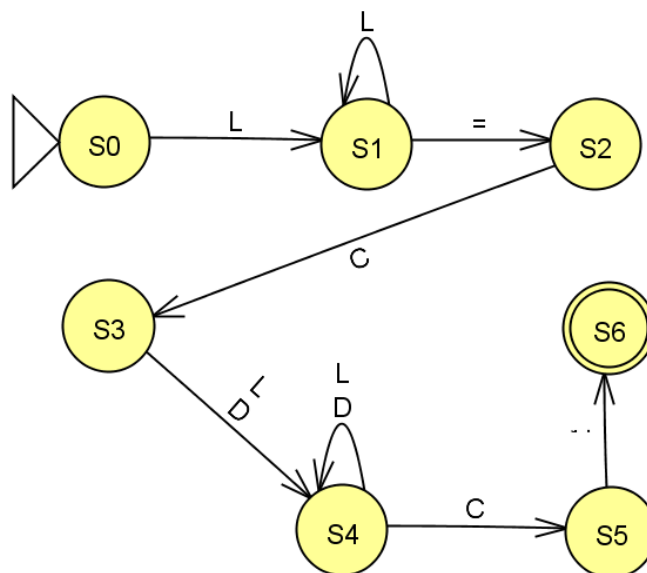
$(L^+ = "C(L|D)^+C";")\$$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
C	3	4,5
L	4	4,5,6
D	5	4,5,6
C	6	7
;	7	8
\$	8	---

	Estado	Valores	Siguientes
	S0	L	$L=\{1,2\}=S1$
	S1	L, =	$L=\{1,2\}=S1$ $"="=\{3\}=S2$
	S2	C	$C=\{4,5\}=S3$
	S3	L,D	$L=\{4,5,6\}=S4$ $D=\{4,5,6\}=S4$
	S4	L,D,C	$L=\{4,5,6\}=S4$ $D=\{4,5,6\}=S4$ $C=\{7\}=S5$
	S5	;	$"="=\{8\}=S6$
Aceptación	S6	\$	

	Estado	L	=	C	D	;
	S0	S1	-	-	-	-
	S1	S1	S2	-	-	-
	S2	-	-	S3	-	-
	S3	S4	-	-	S4	-
	S4	S4	-	S5	S4	-
	S5	-	-	-	-	S6
Aceptación	S6	-	-	-	-	-



Ancho

$L = \{a-z, A-Z\}$

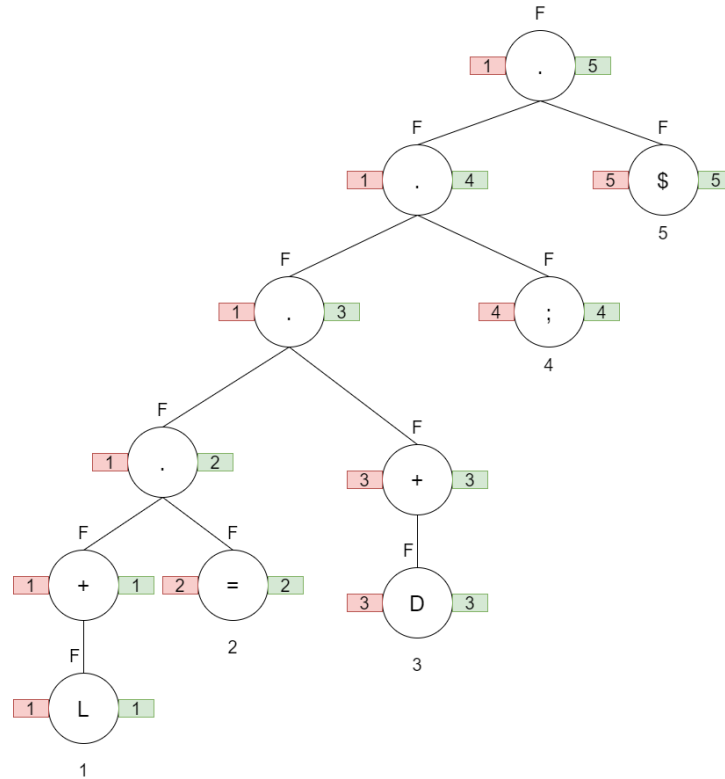
$D = \{0,1,2,3,4,5,6,7,8,9\}$

Expresión Regular:

$L^+ = "D+";"$

Proceso para Autómata Finito Determinista:

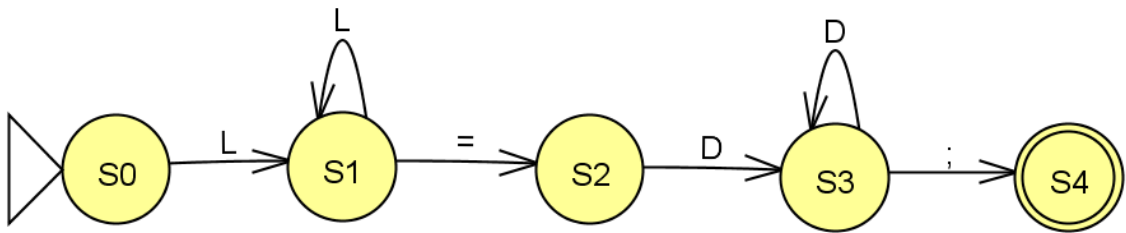
$(L^+ = "D+";")\$$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
D	3	3,4
;	4	5
\$	5	--

	Estado	Valores	Siguientes
	S0	L	$L=\{1,2\}=S1$
	S1	L, =	$L=\{1,2\}=S1$ $"="=\{3\}=S2$
	S2	D	$D=\{3,4\}=S3$
	S3	D, ;	$D=\{3,4\}=S3$ $;"=\{5\}=S4$
Aceptación	S4		

	Estado	L	=	D	;
	S0	S1	--	--	--
	S1	S1	S2	--	--
	S2	--	--	S3	--
	S3	--	--	S3	S4
Aceptación	S4	--	--	--	--



Alto

L= {a-z, A-Z}

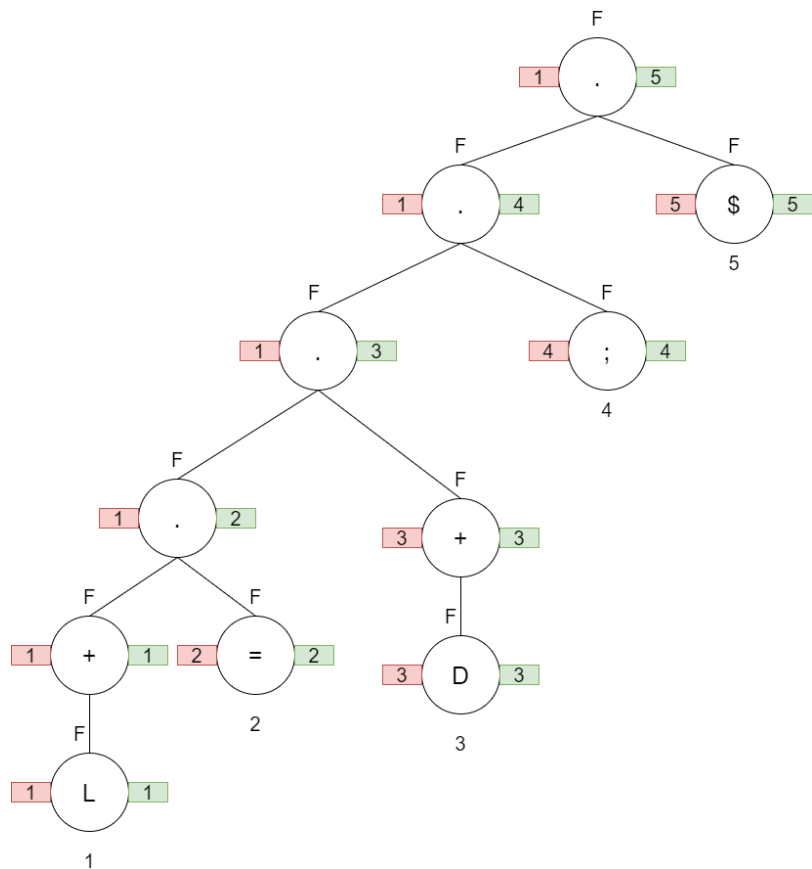
D= {0,1,2,3,4,5,6,7,8,9}

Expresión Regular:

L+=“D+”;

Proceso para Autómata Finito Determinista:

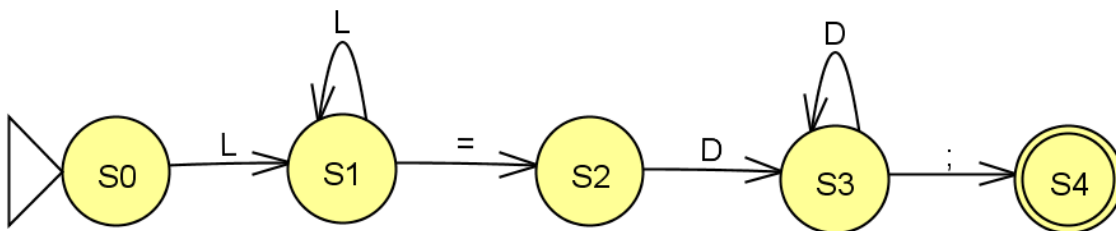
(L+=“D+”;) \$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
D	3	3,4
;	4	5
\$	5	--

	Estado	Valores	Siguientes
	S0	L	$L=\{1,2\}=S1$
	S1	L, =	$L=\{1,2\}=S1$ $"="=\{3\}=S2$
	S2	D	$D=\{3,4\}=S3$
	S3	D, ;	$D=\{3,4\}=S3$ $;\{5\}=S4$
Aceptación	S4		

	Estado	L	=	D	;
	S0	S1	--	--	--
	S1	S1	S2	--	--
	S2	--	--	S3	--
	S3	--	--	S3	S4
Aceptación	S4	--	--	--	--



Filas

$L = \{a-z, A-Z\}$

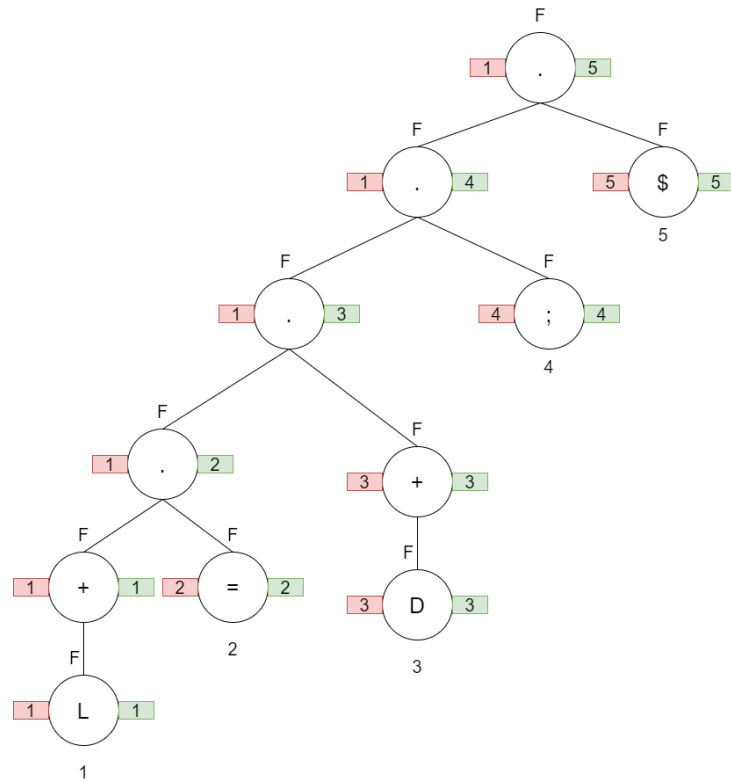
$D = \{0,1,2,3,4,5,6,7,8,9\}$

Expresión Regular:

$L^+ = "D+";"$

Proceso para Autómata Finito Determinista:

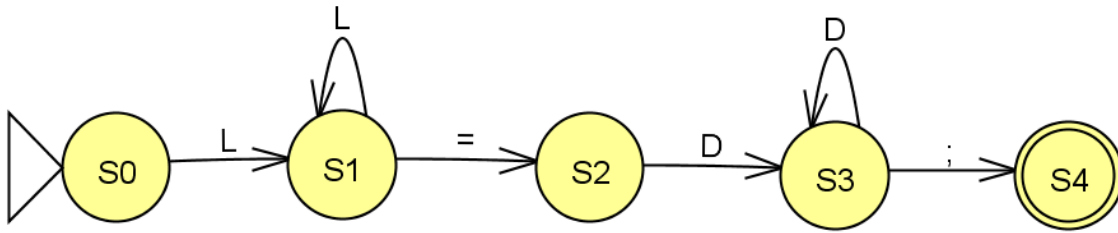
$(L^+ = "D+";)"$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
D	3	3,4
;	4	5
\$	5	--

	Estado	Valores	Siguientes
	S0	L	L={1,2}=S1
	S1	L,=	L={1,2}=S1 "="={3}=S2
	S2	D	D={3,4}=S3
	S3	D,;	D={3,4}=S3 ;={5}=S4
Aceptación	S4		

	Estado	L	=	D	;
	S0	S1	--	--	--
	S1	S1	S2	--	--
	S2	--	--	S3	--
	S3	--	--	S3	S4
Aceptación	S4	--	--	--	--



Columnas

L = {a-z, A-Z}

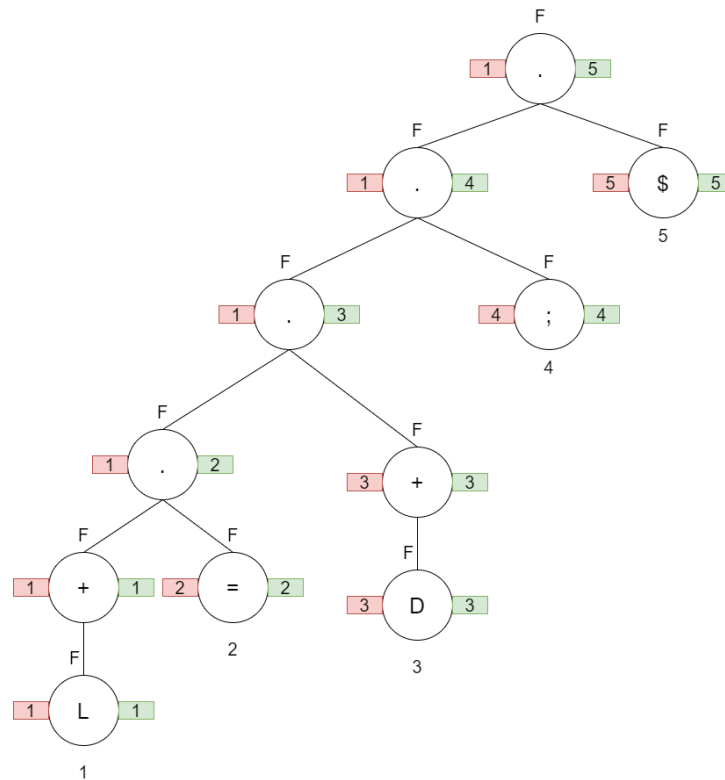
D = {0,1,2,3,4,5,6,7,8,9}

Expresión Regular:

L+ = "D+";"

Proceso para Autómata Finito Determinista:

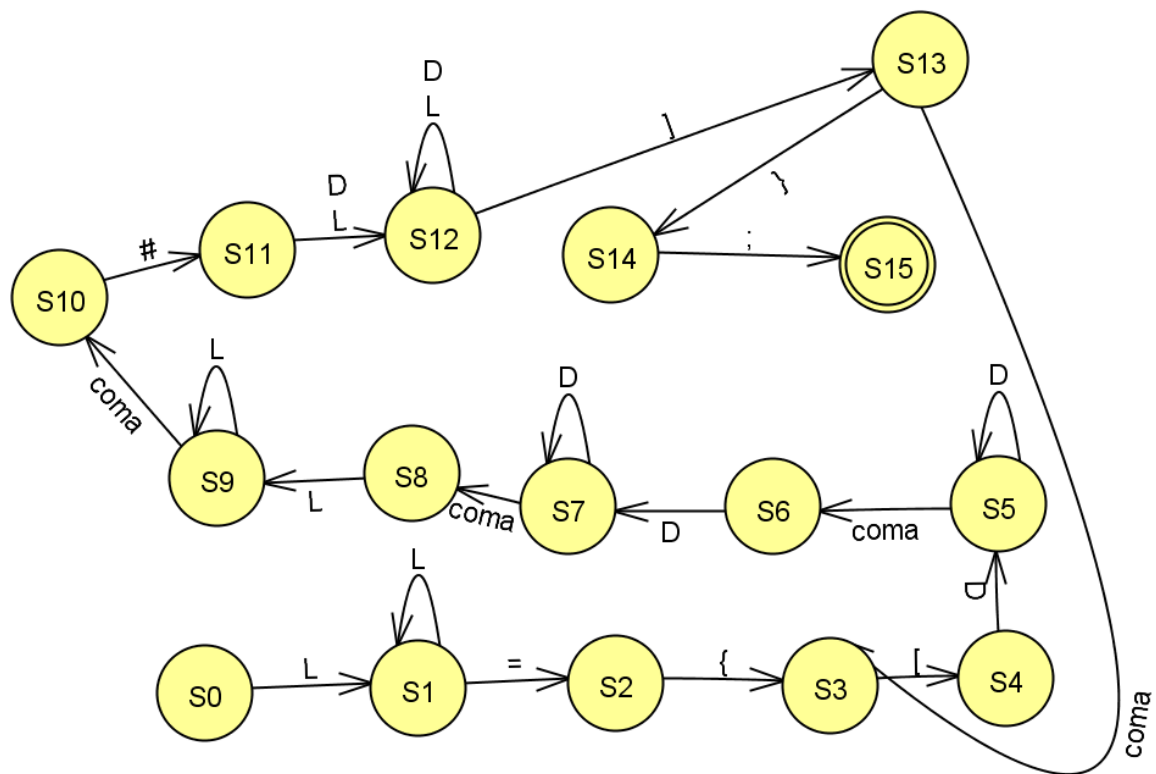
(L+ = "D+";)\$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
D	3	3,4
;	4	5
\$	5	--

]	14	15,4,16
,	15	4,16
}	16	17
;	17	18
\$	18	--

	Estado	Valores	Siguientes
	S0	L	$L=\{1,2\}=S1$
	S1	L,=	$L=\{1,2\}=S1$ $(-)=\{3\}=S2$
	S2	{	$\{=\{4\}=S3$
	S3	[	$[=\{5\}=S4$
	S4	D	$D=\{5,6\}=S5$
	S5	D,	$D=\{5,6\}=S5$ $(.)=\{7\}=S6$
	S6	D	$D=\{7,8\}=S7$
	S7	D,	$D=\{7,8\}=S7$ $(.)=\{9\}=S8$
	S8	L	$L=\{9,10\}=S9$
	S9	L,	$L=\{9,10\}=S9$ $(.)=\{11\}=S10$
	S10	#	$\#=\{12,13\}=S11$
	S11	L D	$L=\{12,13,14\}=S12$ $D=\{12,13,14\}=S12$
	S12	L D]	$L=\{12,13,14\}=S12$ $D=\{12,13,14\}=S12$ $]=\{4,15,16\}=S13$
	S13	, }	$(.)=\{4\}=S3$ $\}=\{17\}=S14$
	S14	;	$:=S15$
Aceptación	S15	\$	



Filtros

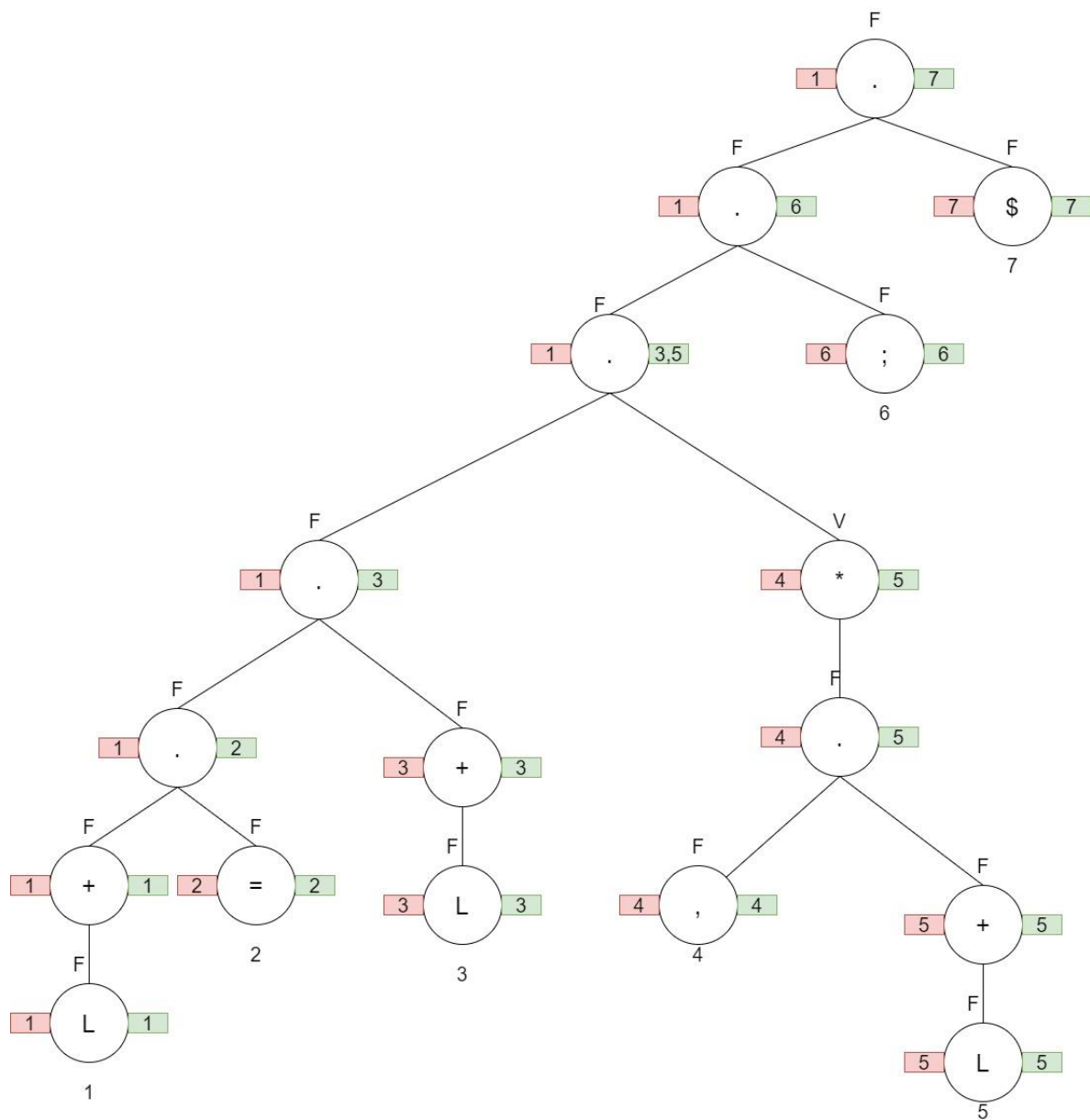
$L = \{a-z, A-Z\}$

Expresión Regular:

$L^+ = "L^+";"$

Proceso para Autómata Finito Determinista:

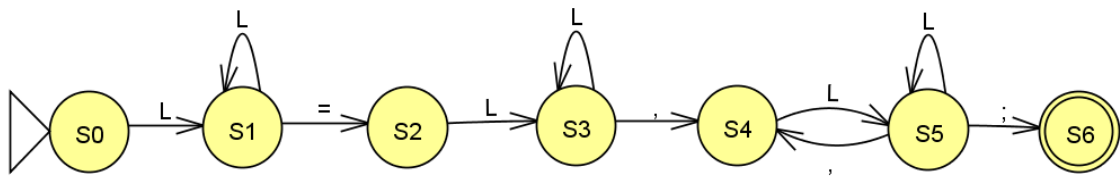
$(L^+(=)(L^+)((,)(L^+))^*(:))\$$



Valor	Hoja	Siguientes
L	1	1,2
=	2	3
L	3	3,4,6
,	4	5
L	5	5,4,6
;	6	7
\$	7	--

	Estado	Valores	Siguientes
	S0	L	$L=\{1,2\}=S1$
	S1	$L =$	$L=\{1,2\}=S1$ $(=)=\{3\}=S2$
	S2	L	$L=\{3,4,6\}=S3$
	S3	$L , ;$	$L=\{3,4,6\}=S3$ $(,)=\{5\}=S4$ $:=\{7\}=S6$
	S4	L	$L=\{5,4,6\}=S5$
	S5	$L , ;$	$L=\{5,4,6\}=S5$ $(,)=\{5\}=S4$ $:=\{7\}=S6$
Aceptación	S6	\$	

	Estado	L	=	,	;
	S0	S1	--	--	--
	S1	S1	S2	--	--
	S2	S3	--	--	--
	S3	S3	--	S4	S6
	S4	S5	--	--	--
	S5	S5	--	S4	S6
Aceptación	S6	--	--	--	--



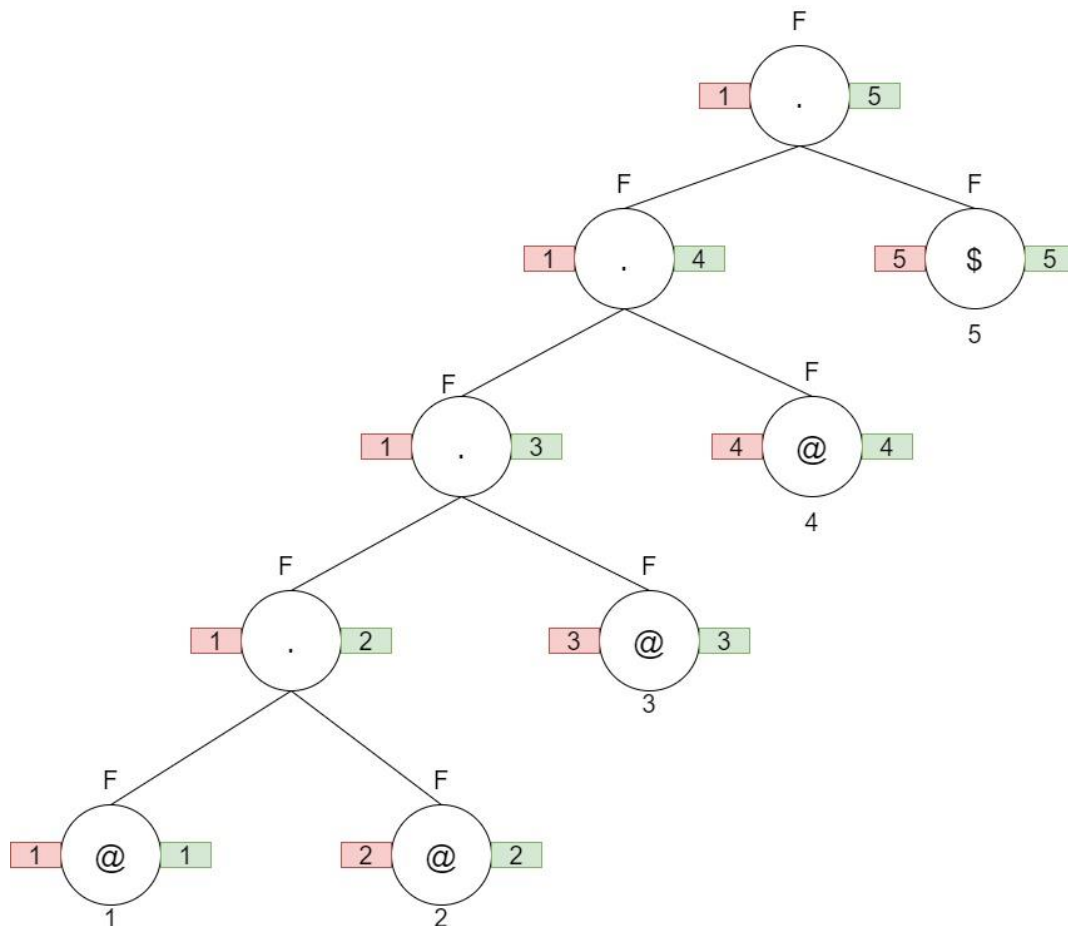
Separador

Expresión Regular:

@@@@

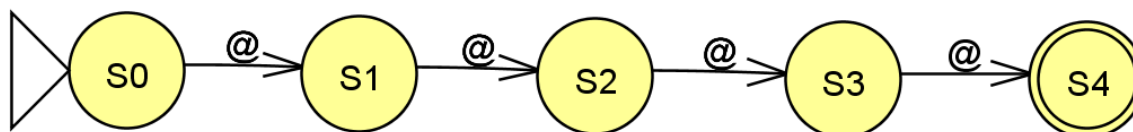
Proceso para Autómata Finito Determinista:

(@@@@)\$



Valor	Hoja	Siguientes
@	1	2
@	2	3
@	3	4
@	4	5
\$	5	--

	Estado	Valores	Siguientes
	S0	@	@={2}=S1
	S1	@	@={3}=S2
	S2	@	@={4}=S3
	S3	@	@={5}=S4
Aceptación	S4	\$	



Durante el desarrollo del código dado que algunos autómatas son igual al inicio se reutilizaron los estados iniciales como el S0, S1, S2 y quedo como resultado el siguiente autómata finito determinista:

